Task 1: feature used

1. MFCC (Mel-Frequency Cepstral Coefficients)
2. Δ MFCC
3. ΔΔ MFCC
4. Chroma
5. Spectral centroid
6. Spectral rolloff
7. Spectral bandwidth
8. ZCR (Zero-Crossing Rate)
9. Tempo

Task 1：model implementation

- Feature Extraction & Pooling

  Compute frame level timbral and spectral features with Librosa and convert variable length sequences into a fixed length vector via time pooling (mean / std / max) per feature.

- Preprocessing

  Standardization with StandardScaler (per feature Z score) to remove scale differences across dimensions;

  Normalization with L2 Normalizer (per sample unit norm) so the classifier focuses on feature direction rather than overall energy.

Task 1 : model implementation

- Classifier

  Employ a Support Vector Machine (SVM) with RBF kernel (C = 2.0, gamma = 'scale') implemented via scikit-learn’s SVC.

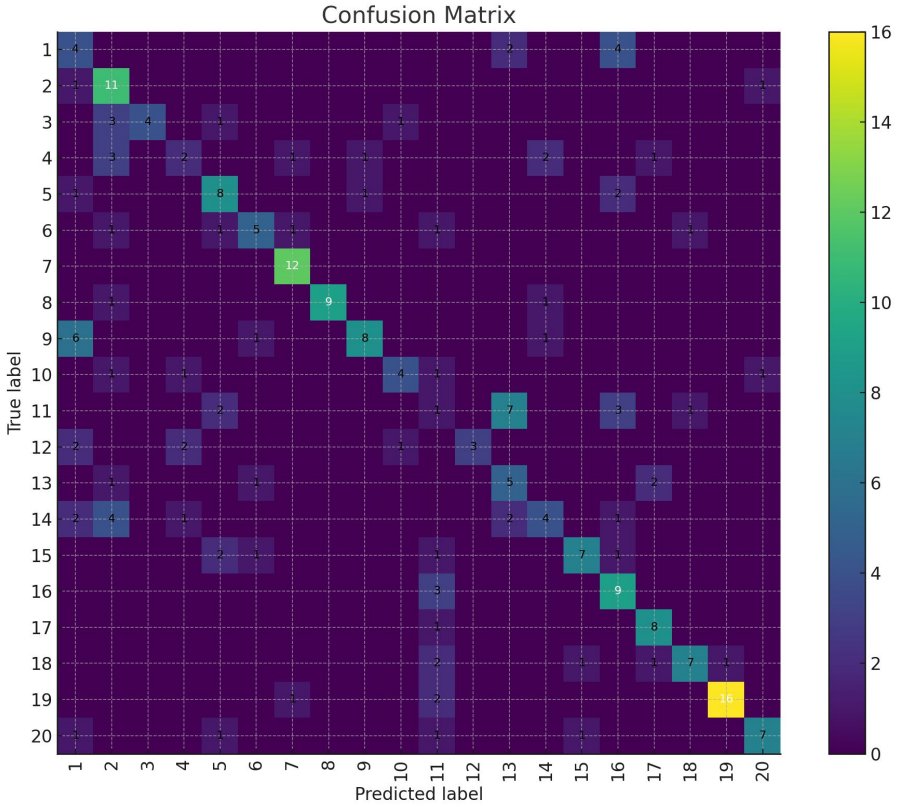
- Training & Evaluation

  A scikit learn Pipeline (StandardScaler → L2 Normalizer → SVM) is fit on the training set and evaluated on the validation set.

Task 1 : validation result

Top-1 accuracy: 0.5801

Top-3 accuracy: 0.8182



Task 2: Pre-processing & Augmentation

Wave cropping

- fixed 16 second clip (num_samples = SR × 16)
- Train: the crop start index is sampled uniformly with torch.randint, per epoch and per DataLoader worker, so each pass sees a different segment.
- Validation: use a center crop

Wave augmentations (train only)

- Random gain: uniformly sampled in [−6, +6] dB
- Additive white noise: applied with 50% probability, SNR uniformly sampled in [15, 30] dB
- After augmentations, clamp the waveform to [−1, 1]

Mel-spectrogram

- Parameters: n_fft = 1024, hop_length = 320, f_max = 7900 Hz, n_mels = 96, power = 2
- standardization: apply $(x - \mu) / \sigma$ to each mel-spectrogram
- SpecAug (train mode only): FrequencyMasking(16) + TimeMasking(32)

Model Architecture — ShortChunkCNN

- Input: (B, T) waveform → log-Mel (B, 1, F, T')
- 7 convolutional blocks
  - Blocks 1–6 use pooling (2, 1), progressively downsample frequency only
  - Block 7 uses pooling (1, 2), downsample time
- After convs, the frequency dimension is 1, then we squeeze to (B, C, T'')
- Temporal pooling: x = (GAP(x) + GMP(x)) to aggregate over time
- Classifier head: Linear(1024→512) → BN1d → ReLU → Dropout(0.5) → Linear(512→20)

Training

- Loss: cross entropy with label smoothing = 0.03
- Optimizer: AdamW
- LR schedule: OneCycleLR with max_lr=3e-3, epochs=30, pct_start=0.15, div_factor=10, final_div_factor=25, steps_per_epoch = len(train_loader)
- Batch size: 16
- Mixed precision: PyTorch AMP (autocast + GradScaler)
- Gradient clipping: 1.0 (global norm)
- Shuffling & seeds

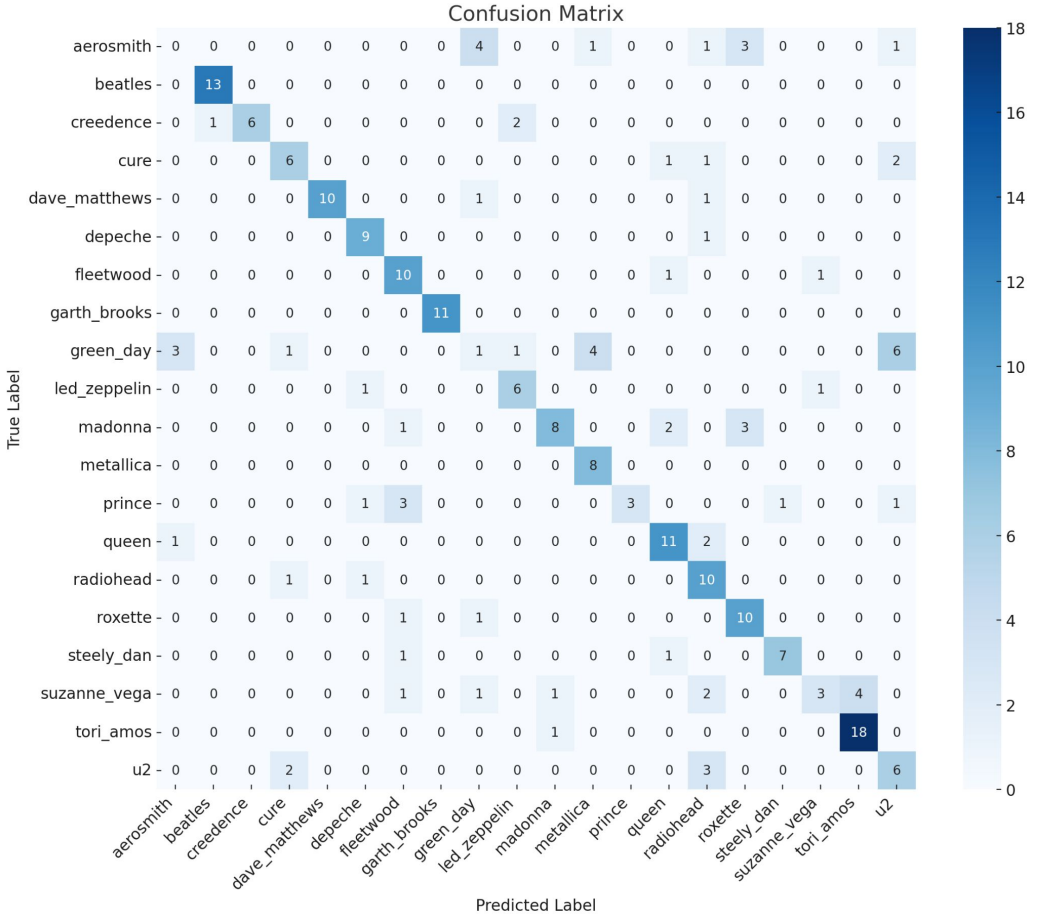
Validation&Inference

- Crops: Each track is split into 8 s windows with 4 s hop
- Forward & aggregation: For a song’s crops, run the model in mini-batches (16), apply softmax, then average probabilities over all crops
- GPU & AMP: validation runs on GPU, with AMP enabled when CUDA is available
- Averaging across segments reduces variance from local, non-representative audio and yields a robust song-level prediction
- Both of them used the same method to get the answer

Task 2 :validation result

val top1: 0.6753

val top3: 0.8139



https://drive.google.com/drive/u/0/folders/10A-tlN0tqtnEYUtTx7fBVZI-mi5NguVQ